

Docker is a containerization platform that packages applications with their dependencies so they run consistently across environments.

Images: - template of project - clone of project, each and everything - A Docker image is a read-only template used to create containers.

Container - running instance of images

- Kind of ready to use the software
- Read-only template
- Images can not be updated
- Need to make changes in the image, then need to create new images
- Images can not run directly

A container is a lightweight, isolated runtime created from a Docker image.

Image vs Container?

An image is a blueprint; a container is a running instance of that blueprint.

Image

- Blueprint
- Read-only
- Never changes
- Built once

Container

- Running instance of image
- Has its own filesystem
- Can read external files
- Can restart processes

Image example

Application tool and libraries

Node.js setup

React set up

Dependency

Code

Supporting tools if we have

Dockerfile too - important

So we can say the image is a template

Docker container

- It is a runnable instance of an image, like an object in classes
- Container is a process run for the application help using images
- Isolated process
- Run independently on the user's system

Docker Hub & Docker Desktop

Docker Hub: an online tool we can use on a browser

- Docker Hub is a **cloud registry** used to **store** and **share Docker images**.
- Here we also have parent images - any base images like for a React application, basic Node images, React setup project images, etc
- **Docker Hub** is an **online website** where Docker images live.

Think of Docker Hub like:

- GitHub → for code
- Docker Hub → for **Docker images**

Docker Desktop

Docker Desktop is a **GUI application** that includes Docker **Engine** and **tools** to **manage containers** locally.

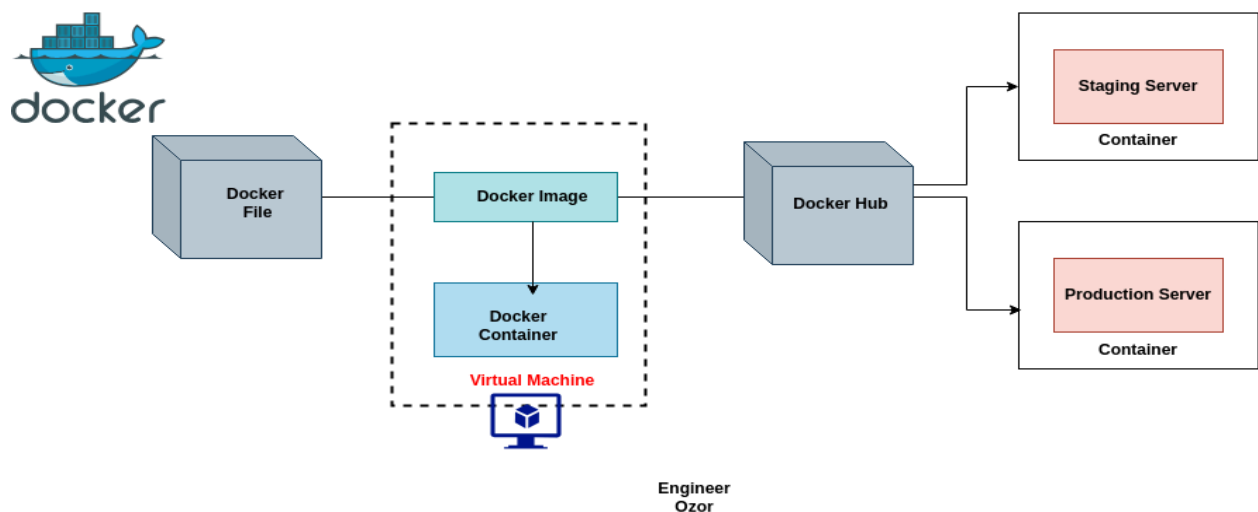
- A **graphical interface** (click buttons instead of typing)
- Docker Engine (the core Docker)
- Docker Compose
- Logs, stats, container list
- Easy start/stop

Docker → runs containers

Docker Desktop → app to manage Docker visually

Docker Hub → website to download/upload images

Docker Workflow



Base Image or Parent Images

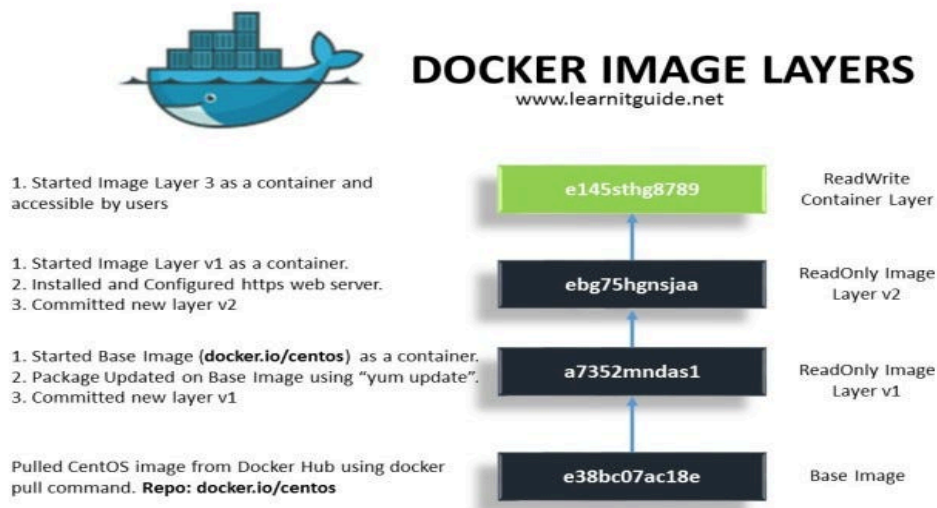
Sometimes projects need another image, or another library or other programming language

A **base image** (also called a **parent image**) is the **starting image** you build on top of.

A base image is the parent image on which a Docker image is built.

These usually come from **Docker Hub**:

- `ubuntu`
- `alpine`
- `python`
- `node`
- `openjdk`
- `Nginx`



Pull the image and run the container

To pull the official Node image from **Docker Hub**, run:

```
docker pull node
```

```
docker pull node:18
```

Or directly from Docker Desktop, find the image for Node and click on the pull button after installing it to show the image in the images module, and we can run it from here, and also using the command line

```
docker run -it node
```

or

```
docker run -it node /bin/bash - this is for logs and error handling
```

Docker pull - It downloads an image from Docker Hub to the local machine.

docker run - **It creates and starts a container from a Docker image.**

Difference between `docker run` and `docker start`?

`docker run` creates + starts a container; `docker start` starts an existing stopped container.

Dockerfile

A Dockerfile is a text file containing instructions to build a Docker image.

FROM do in Dockerfile? - It specifies the base image for the Docker image.

Create a Dockerfile in the project

`touch dockerfile`

For example react project

#1 Base image

`FROM node:18-alpine`

#2 Set working directory

`WORKDIR /app` - optionally, if you create this then you can run your whole code in docker in app directory

#3 Copy package files first (for cache)

`COPY` source destination - explanation of line

`COPY package*.json ./` optionally you can run the code

```
COPY . .
```

#4 Install dependencies

`RUN npm install`

#5 Copy rest of the project

`COPY . .`

#6 Build Next.js app

`RUN npm run build`

#7 Expose Next.js port

`EXPOSE 3000`

#8 Start the app

`CMD ["npm", "run", "dev"]`

Example

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
```

```

COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "run", "dev"]

```

- You should **NOT** copy `.env` and **NOT** copy `node_modules`

touch .dockerignore

- `node_modules`
- `.env`
- `.git`
- `.gitignore`
- `Dockerfile`
- `README.md`

If we do not need to add any specific extension file in the docket thne we can add it in the **.dockerignore file**

***.txt**

Noe any .txt extension files are not added in Docker

Run this in the terminal - `docker build -t next-admin-panel .`

docker - docker command

build - create build

-t - we apple tag

Next-admin-panel - tag for name this name use

. - path '.' means root folder, you can add /app or /src

It will create a build and create an image in the Docker Desktop image module, which you can view

Now run the container and create

Create using Docker Desktop - apply name and port (if you add port 0 thne it will generate a random port)

Using command line - **docker run -d -p 4000:3000 --name admin-ui test-airecruitor**

-Here - 3000 is your system project run port, and 4000 is the new port you give for - -

-Docker tun on **-this port**

-admin-ui is the **container name**

-Test-airecruitor is the **image name**

- For container log view command - **docker logs admin-ui**

For .env data pass

From the **same folder** where **.env** exists, run:

```
docker run -d -p 4000:3000 --env-file .env --name admin-ui test-airecruitor
```

Docker, read my **.env** file and give those variables to the container

Check if the env actually reached the container

Command - **docker exec -it admin-ui env**

or

```
docker exec -it admin-ui sh
```

Now inside the container, run:

```
env
```

It will show like this

```
docker exec -it admin-ui sh
```

```
# env
```

```
NODE_VERSION=25.4.0
```

```
HOSTNAME=4c7d5f6ac585
```

```
YARN_VERSION=1.22.22
```

```
HOME=/root
```

```
NEXT_PUBLIC_RAZORPAY_KEY_ID=
```

```
NEXT_PUBLIC_GOOGLE_OAUTH_CLIENT_ID=
```

```
RECAPTCHA_SECRET=
```

```
TERM=xterm
```

```
NEXT_PUBLIC_ENV=development
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
NEXT_PUBLIC_CD_API_URL=https://dev-cd-api.airecruitor.com/
```

```
NEXT_PUBLIC_RESUME_SCREENING_API_URL=https://dev-resume-score.airecruitor.com/
```

```
NEXT_PUBLIC_API_URL=https://dev-api.airecruitor.com
```

```
PWD=/
```

```
NODE_ENV=development
```

```
NEXT_PUBLIC_RECAPTCHA_SITE_KEY=-
```

```
#
```

How do you Dockerize a Next.js app?

By creating a Dockerfile, building the app, and running it inside a Node-based container.

Why do you copy **package.json** first?

To leverage Docker cache and avoid reinstalling dependencies on every build.

What does **EXPOSE 3000** do?

It documents the port the container listens on.

Why use `node:18-alpine`?

It's lightweight, secure, and reduces image size.

How do you run a Dockerized Next.js app?

Using `docker run -p 3000:3000 image-name`.

Can this image run on any server?

Yes, anywhere Docker is installed.

Should `node_modules` be copied into Docker image?

No, dependencies should be installed inside the container.

Why not copy `.env` file?

Because it contains secrets and should not be baked into the image.

How do you pass environment variables to Docker?

Using `--env`, `--env-file`, or Docker Compose.

What is `.dockerignore`?

A file that prevents unnecessary or sensitive files from being copied into the Docker image.

What happens if you copy `node_modules`?

It increases image size and may cause OS compatibility issues.

How are environment variables handled in [Next.js](#)?

Public variables use `NEXT_PUBLIC_*` and must be available at build time.

Delete images from the command line

`docker image rm image-name`

`docker image rm image-name -f` - it will delete used images also

`docker ps -a` - return running containers

`docker container rm container-name`

`docker container rm container-name -f` - it will delete used containers also

If Docker has a **different network world**,

why can I access my containers on

`localhost:3000`, `localhost:8000`, etc

- Docker containers **do NOT** automatically run on localhost.
- They run in their **own private network**.
- They appear on `localhost` **ONLY** because of this line:

ports:

- `"3000:3000"`

Means:

“Docker, please open **port 3000 on my HOST**
and forward all traffic to **port 3000 inside this container.**”

Browser



localhost:3000



Docker port forward (NAT)



frontend container:3000

Your container is **still not on localhost**.

Docker is just acting as a **router**.

If you run Docker on a **local machine** and give a port → it runs on

localhost:port

If you run Docker on a **server** → it runs on **server_ip:port**

The precise version

When you publish a port, Docker exposes the container on the IP of the machine where Docker is running.

Docker Volume

A Docker volume is a **place to store data outside the container**.

A Docker volume is a **storage location that lives OUTSIDE the container**.

Continuous changes in the container with volume

Why does it need - Because containers are temporary.

Containers can be deleted. Volumes survive.

Images don't change. Containers + bind mounts do.

Syntax: **-v SOURCE: TARGET**

Without volume:

- Container stops
- Container deleted
- All data lost

With volume:

- Container stops
- Container deleted
- Data remains safe

Types of storage in Docker

1. Docker Volume (Docker-managed)

-v mydata:/app/data

- Stored inside the Docker system
- Used in production
- For databases, uploads, logs

Command -

```
docker run --name my-app \ # container name
-p 4000:3000 \             # host: container port
-v mydata:/app/data \      # Volume Name
Image-name                 # image
```

2. Bind Mount (Host folder)

-v \$(pwd):/app

- Maps your local folder
- Used in development
- For live code changes

This is NOT a “volume” technically — it’s a **bind mount**.

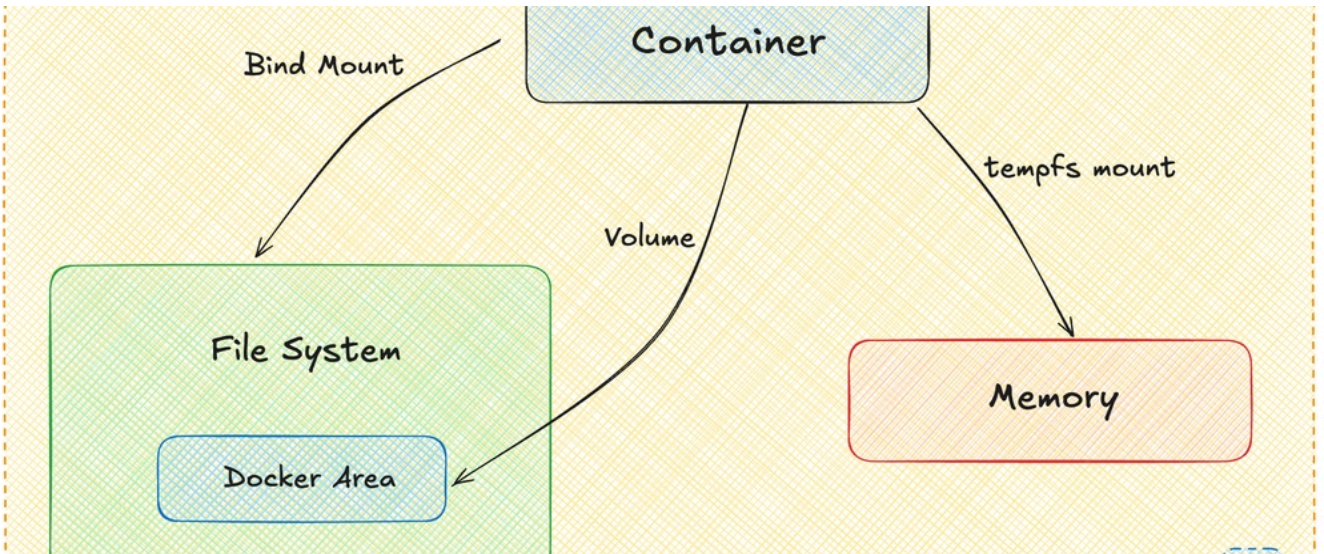
Bind Mount = Live Reload // Like nodemon run data changes on the system and container auto changes also

Works in development mode, not in production mode

Command -

```
docker run --name container-name -p 4000:3000 -v systemProjectPath:containerProjectPath
image-name
```

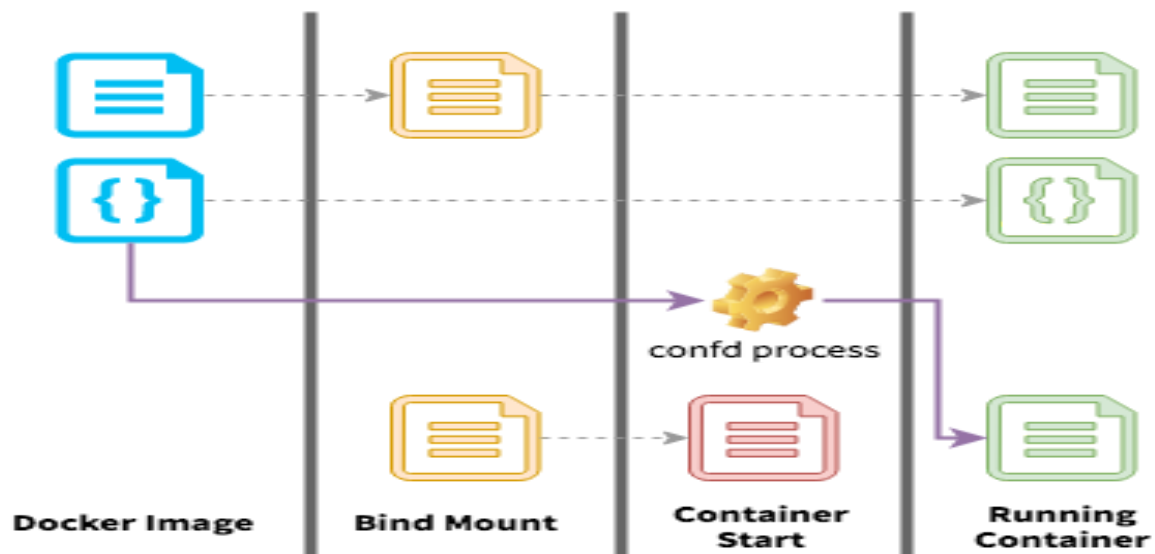
```
docker run
--name container-name      # container name
-p 4000:3000               # host: container port
-v systemProjectPath:/app  # Bind mount
image-name                 # image
```

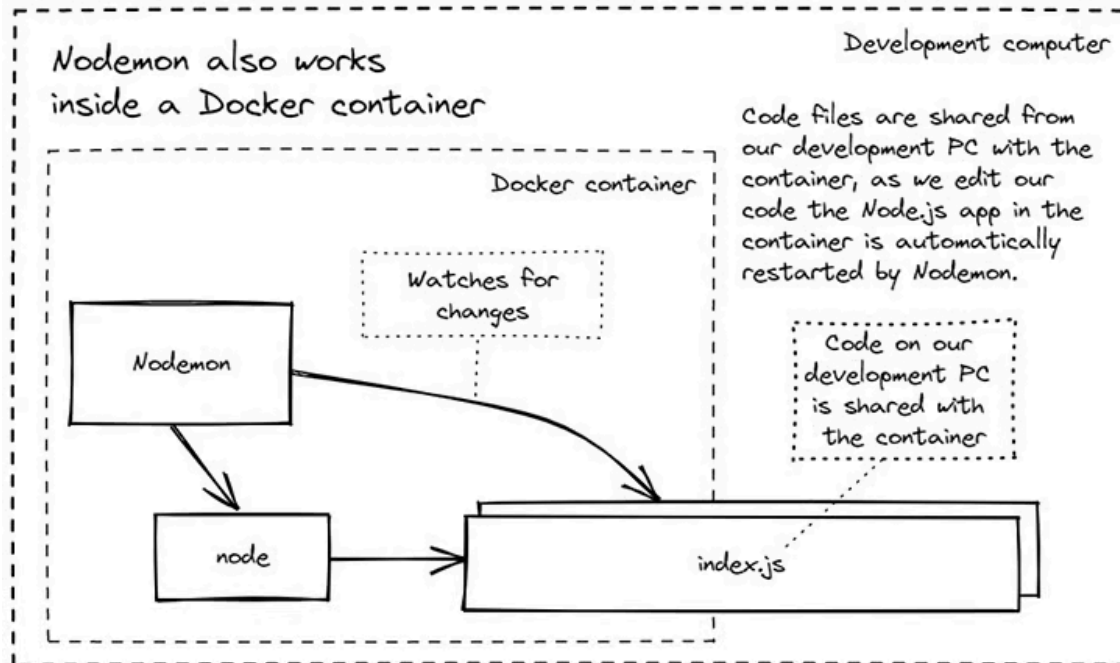


Docker volume (bind mount) + nodemon = live reload

What's happening:

- Your code lives on your laptop 🖥️
- Docker container reads that same code via volume
- Nodemon watches file changes
- Container restarts the app automatically





How to know if it's a volume or a bind mount?

SOURCE value	Type
mydata	Docker volume
/home/user/app	Bind mount
\$(pwd)	Bind mount

Syntax to mount a volume? : -v volume_name:/container/path

Syntax for bind mount? : -v /host/path:/container/path

Which is used for live reload ? : Bind mount

Which is used for persistent data? : Docker volume

Can multiple volumes be attached? - Yes

Syntax for multiple volumes:

```
docker run \
-v volume1:/path1 \
-v volume2:/path2 \
image-name
```

Example (bind mount + volume together): docker run --name my-app \

```
-p 3000:3000 \
-v $(pwd):/app \
-v app_data:/app/data \
node:18
```

docker volume rm db_data - using this command we can delete volume

Can volume and bind mount be used together? Yes, both can be used in the same container.

Is there a limit to the number of volumes? No practical limit; depends on system resources.

Why is bind mount used instead of a volume for live reload?

Because bind mounts sync host files in real time.

What happens to the image code when a bind mount is used?

It is overridden at runtime by host files.

Production Docker setup uses?

No bind mounts, no nodemon, only volumes for data.

How do you enable live reload in Docker?

Using bind mounts and a file watcher like nodemon.

Does Docker volume update the image? No, images are immutable.

Volume vs bind mount for live reload? Bind mount is used for live reload, not volume.

Why not use bind mounts in production? Why not use bind mounts in production?

Do code changes require rebuilding the image in dev? No, bind mounts avoid rebuilding.

Do code changes require rebuilding in prod? Yes, the image must be rebuilt.

Docker Hub

Docker Hub is a cloud registry where you:

- Store Docker images
- Share them with others
- Pull images from anywhere

Think:

GitHub for Docker images

How to Share an Image on Docker Hub

docker images - check existing images

1. Log in to Docker Hub

docker login

2. Tag your image (IMPORTANT)

docker tag local-image yourusername/repo-name:tag

Example: docker tag next-admin-pane:latest myuser/next-admin-panel:1.0

docker tag next-admin-panel:v2 divyatec/hrms-panel:1.0

First I created hrms-pane name repo as private and in my local image have version v2, so I added next-admin-panel:v2

3. Push image to Docker Hub

docker push myuser/next-admin-panel:1.0

4. Pull the image to Docker Hub

docker pull divyatec/hrms-panel:1.0

How to create a private repo

On Docker Hub:

1. Create a new repository
2. Choose Private
3. Push image

Private Repository

- Only you (or allowed users) can pull the image
- Login is required
- Used for company code/admin panels / internal apps

Image Size & Limits

Docker Hub does not enforce a strict “image size limit”, but limits depend on account plan and fair usage policies.

In practice:

- Images can be **hundreds of MBs**
- Even **1–2 GB images** are possible (not recommended)
- Smaller images = faster pulls & better practice

Docker Compose

Why Docker Compose

- A real application usually has multiple services, like a frontend, backend, and database.
- Without Docker Compose, each service must be started separately using multiple **docker run** commands.
 - Too many commands to remember
 - Port mappings are easy to forget
 - Environment variables are hard to manage
 - Stopping all services is inconvenient

Docker Compose solves this by letting you define and run all services together using a single configuration file and one command.

Docker Compose itself **does nothing without a file**.

What is **Docker Compose**?

Docker Compose is a TOOL (command-line tool) that:

- Reads a config file
- Creates containers
- Starts them together
- Connects them automatically

Command : docker compose up

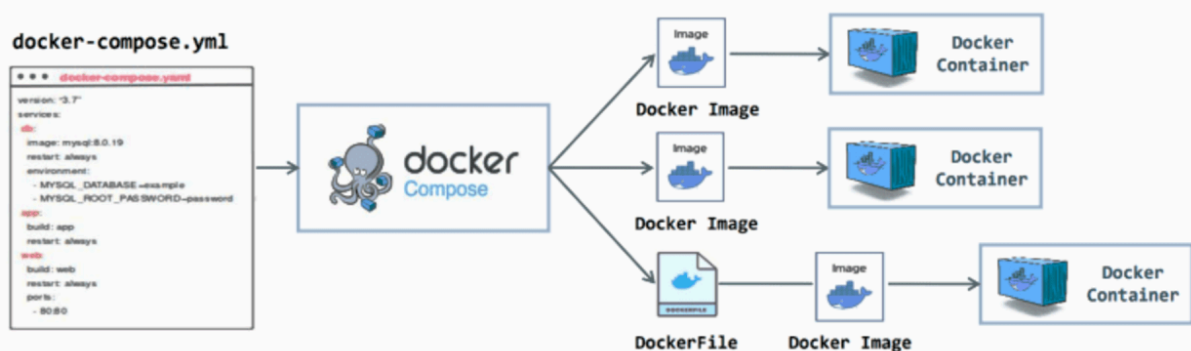
What is **docker-compose.yml**?

docker-compose.yml is just a **YAML configuration file**.

- It tells Docker Compose:
- Which containers to run
- Which images to use
- Ports, volumes, env variables
- How containers talk to each other

The file does **NOT** run anything by itself.

Docker Compose = tool
docker-compose.yml = instructions



Docker-compose.yml file
services:

```
service-name:
  image: image-name
  ports:
    - "host:container"
```

Example with MULTIPLE containers (REAL WORLD)

```
services:
  frontend:
    image: divyatec/hrms-panel:1.0
    ports:
      - "3000:3000"

  database:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
```

docker compose up
docker compose down

Steps and all thing which compose.yaml file have to know

Term	Meaning
service	One container
image	Docker image to run
build	Build image from Dockerfile
volume	Persistent storage
network	Communication between services
env	Environment variables

BASIC STRUCTURE OF `compose.yaml`

```
services:
  service-name:
    image: image-name
```

That's the minimum valid Compose file.

1. `services` (THE HEART OF COMPOSE)

```
Services:
  frontend:
    image: nginx
```

- `services` is mandatory
- Each service = one container
- Service name can be anything

2. `image` vs `build`

Using an image (already built)

```
services:
  app:
    image: node:18
```

Building from Dockerfile

```
services:
  app:
    build: .
    OR
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
```

Use `build` when you own the code.

3. **ports** (Expose app to browser)

```
a.
b.   ports:
c.     - "3000:3000"
d. -----
e.   Format
f.     HOST:CONTAINER
g. -----
h.   Example
i.     services:
j.       web:
k.         image: nginx
l.         ports:
m.         - "8080:80"
n.
```

4. **environment** (ENV variables)

```
Inline env Vars
environment:
  NODE_ENV: production
  PORT: 3000
  OR

Using .env file (BEST PRACTICE)

env_file:
- .env
```

.env stays outside the image → secure.

5. **volumes** (MOST IMPORTANT FOR REAL PROJECTS)

Named volume (production data)

```
volumes:
  - app_data:/app/data
```

Bind mount (development / live reload)

```
volumes:
  - ./app
```

Full example

```
services:
  app:
    image: node:18
    volumes:
      - ./app
```

6. Top-level volumes section (IMPORTANT)

```
volumes:
  app_data:
```

OR

```
services:
  db:
    image: mysql:8
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

7. **depends_on** (startup order)

```
depends_on:
  - db
```

Full Example

```
services:
  backend:
    image: node:18
    depends_on:
      - db

  db:
    image: mysql:8
```

Controls start order, not readiness.

8. Networking (AUTOMATIC but IMPORTANT)

Default behavior:

- Compose creates one network
- Services talk using the service name
- Creates a private network
- Connects all services to it
- Allows containers to talk using service names

```
services:
  backend:
    image: node
  db:
    image: mysql
```

Inside backend:

DB_HOST = db

Custom network (advanced)

```
networks:
  app_net:

services:
  app:
    networks:
      - app_net
```

Important: Use service name, not `localhost`

Docker:

- Creates a network automatically
- Creates DNS automatically
- Handles IPs automatically

You just use names.

Docker Compose puts all containers on one private network and lets them talk using service names instead of IP addresses.

How do containers talk in Docker Compose?- Using service names on a shared network.

Why not use localhost? - Because containers are isolated.

Do I need to create a network? - No, Docker Compose does it automatically.

9. `command` (override CMD)

command: `npm run dev`

Useful for:

Dev mode
Custom startup

10. `restart` (production behavior)

restart: `always`

Options:

- `no`
- `always`
- `unless-stopped`
- `on-failure`

11. `container_name` (optional)

container_name: `my-app`

Note: Avoid in large projects (name conflicts).

12. `version` (IMPORTANT HISTORY)

version: `"3.8"`

Note: Modern Docker (v2+) ignores version

13. FULL REAL-WORLD EXAMPLE (Node + DB)

```
services:
  backend:
    build: .
    ports:
      - "3000:3000"
    env_file:
      - .env
    volumes:
      - ./app
    depends_on:
      - db

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

MOST USED Docker Commands

- **List running containers**
docker ps
- **List all containers**
docker ps -a
- **Run a container**
 - docker run image-name
 - **Example:** docker run -d -p 3000:3000 nginx
 - Container runs **in the background**
 - Terminal is **free**
 - App keeps running
 - Logs are not shown automatically
 - **Without -d** → foreground app
 - **With -d** → background service

- **Stop a container**
docker stop container-name
- **Start a stopped container**
docker start container-name
- **Restart a container**
docker restart container-name
- **Remove a container**
docker rm container-name
- **Force remove (even if running):**
docker rm -f container-name
- **List images**
docker images
- **Build image**
docker build -t image-name .
- **Pull image from Docker Hub**
docker pull image-name
- **Remove image**
docker rmi image-name
- **View container logs**
docker logs container-name
- **Follow logs live:**
docker logs -f container-name
- **Go inside a running container**
docker exec -it container-name bash
- **Start services**
docker compose up
Background: docker compose up -d
- **Stop & remove containers**
docker compose down
- **Rebuild images**
docker compose up --build
- **View compose logs**
docker compose logs
- **List volumes**
docker volume ls
- **Inspect volume**
docker volume inspect volume-name
- **Remove volume**
docker volume rm volume-name
Deletes data permanently.
- **List networks**
docker network ls

- **Inspect network**
docker network inspect network-name
- **Cleanup Commands (IMPORTANT)**
Remove unused containers, images, volumes
docker system prune

Check **docker-compose YAML file size**

Linux / macOS

ls -lh docker-compose.yml or ls -lh compose.yml

To see size in bytes only:

stat docker-compose.yml

Check **Docker image sizes**

List all images and their sizes:

docker images

Images used by your compose file only

docker compose images

(or **docker-compose images** on older versions)

See all containers (including stopped)

docker ps -a -s

Disk usage summary (quick health check)

docker system df

(Optional) Find what's eating space inside a container

docker exec -it <container_name> sh

du -h --max-depth=1 /

Useful for:

- log files
- cache folders
- temp files

- ls -lh docker-compose.yml **# compose file size**
- docker images **# image sizes**
- docker compose images **# images used by compose**
- docker ps -s **# container sizes**
- docker system df **# overall disk usage**

Docker Logs OR central Log Management

WHY Loki + Grafana exist

Docker logs alone are NOT enough

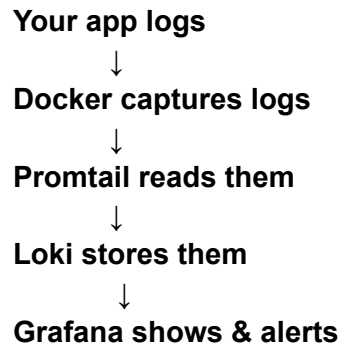
Docker logs:

- live **only on the server**
- are lost if the server dies
- cannot alert you
- are hard to search across containers
- are not historical

How do we collect logs from MANY containers and keep them safe, searchable, and alertable? -

Centralized logging

That's where **Loki + Grafana** come in.



Each tool has one job.

1. Loki (log database)

Loki = log storage engine

- not like MySQL
- not full-text indexing
- cheap and fast

It stores logs like:

timestamp + labels + message

Example:

{service="api", level="error"} DB timeout

Grafana Loki is a **log aggregation system** designed by Grafana Labs.

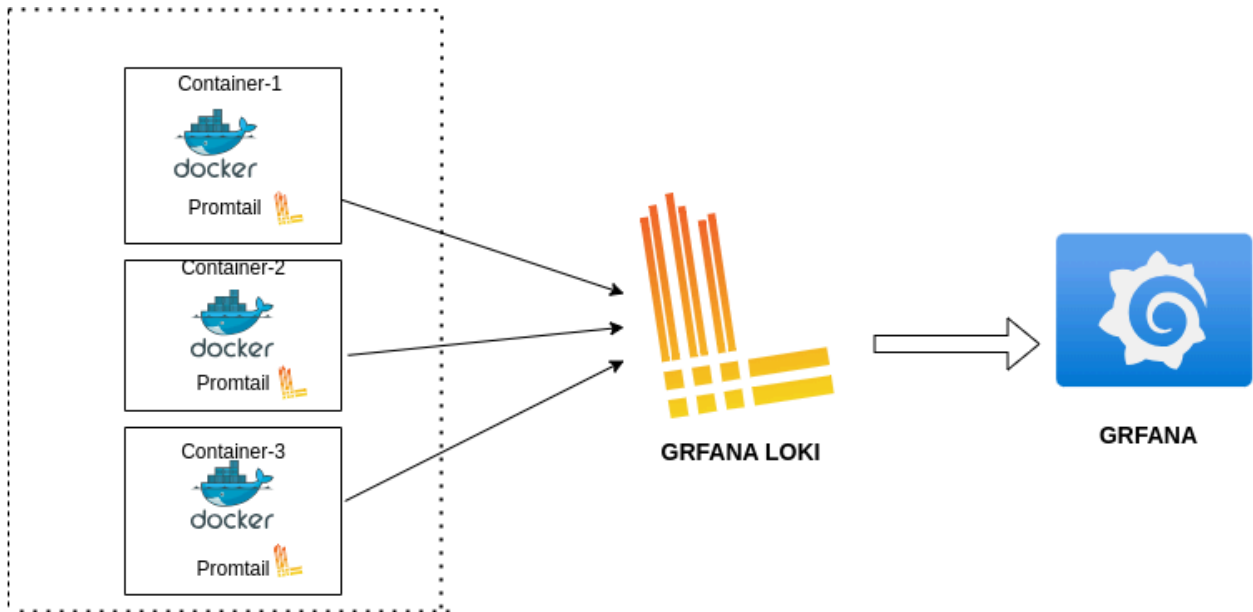
It is **commonly run inside Docker containers** and used in **containerized / microservices environments**.

In simple words

- Loki **collects logs**
- Stores them **efficiently**
- Let's you **query logs**
- Works very well with **Docker, Kubernetes, and Grafana**

Unlike traditional logging tools, **Loki does NOT index log contents**.

It only indexes **labels (metadata)** → which makes it **faster and cheaper**.



Core Components

1 Promtail (Log Collector)

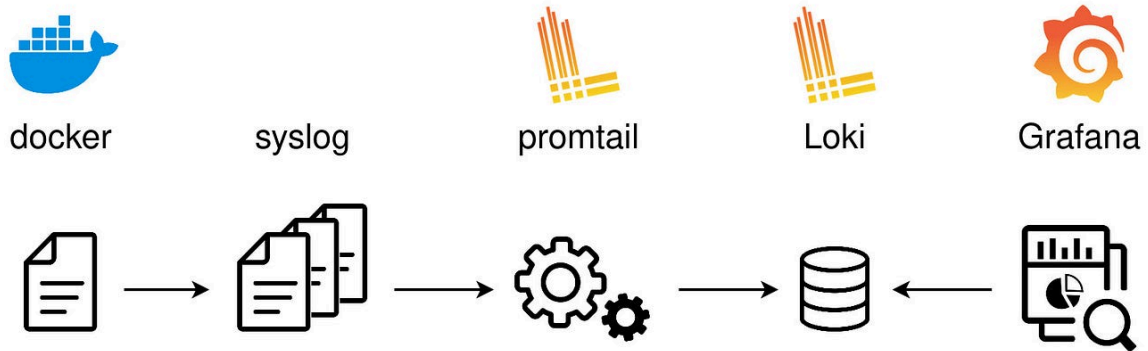
- Runs as a **Docker container**
- Reads logs from:
 - Docker containers
 - `/var/log`
- Adds **labels** (like container_name, app, environment)
- Sends logs to Loki

2 Loki Server

- Receives logs from Promtail
- Stores logs in:
 - Local filesystem
 - S3 / GCS / MinIO (production)
- Indexes **labels only**

3Grafana

- Used to **visualize and query logs**
- Uses **LogQL** (Loki Query Language)



Typical Flow

1. Docker containers generate logs (stdout/stderr)
2. Promtail reads Docker logs
3. Promtail sends logs to Loki
4. Grafana queries Loki and shows logs

Why Loki is great for Docker

Lightweight
Label-based (perfect for containers)
Easy Docker Compose setup
No full-text indexing → low cost

Important Loki Concepts

Labels

- Key-value metadata
- Example:
`{app="backend", container="auth-service"}`

LogQL

- Query language for Loki
- Example:
`{app="nginx"} |= "error"`

Streams

- Unique combination of labels
- Loki stores logs as streams

Exactly how logs appear in Loki, step by step, and specifically what this means:

```
{app="backend", container="auth-service"}
```

Think of this as how Loki “sees” your logs.

How Logs Should Look in Loki

In Loki, logs are shown as:

Log message + labels (metadata)

- Loki does NOT care about log files like traditional systems.
- It groups logs by labels, then stores the actual log lines under them.

Note: Means if I have a container, and in the container my website has any console.log or console.err ?

Yes, but only if the logs go to stdout/stderr — which is exactly what `console.log` and `console.error` do.

If my website runs inside a Docker container and logs using console.log or console.error, Docker captures those logs as stdout and stderr. Promtail reads them and sends them to Loki, where they can be queried using labels in Grafana.

console.log / console.error → Docker stdout/stderr → Promtail → Loki → Grafana

Will Loki add these logs?

YES — Loki will collect them automatically IF:

1. Your app writes logs to:
 - `stdout` → `console.log`
 - `stderr` → `console.error`
2. Promtail is configured to read **Docker container logs**

Step 1: Docker Container Generates Logs

Your container prints logs to **stdout/stderr**:

Docker captures them

Docker automatically captures:

- `stdout`
- `Stderr`

And stores them in a JSON log file.

Step 2: Promtail Reads Docker Logs

Promtail:

- Reads Docker log files

- Extracts metadata (container name, image, labels)
- Adds **custom labels** (like app, env)

```
• app = backend
• container = auth-service
• env = production
```

Step 3: Logs Are Sent to Loki as a Stream

In Loki, logs are stored as **streams**.

What is a Stream?

A **stream** = unique label combination

So this becomes **ONE stream**:

```
{app="backend", container="auth-service", env="production"}
```

Inside that stream, Loki stores log lines **in order**:

```
2026-01-30 10:01:12 User login successful
2026-01-30 10:01:15 Invalid password for user admin
2026-01-30 10:01:20 JWT token expired
```

Breaking Down {app="backend", container="auth-service"}

This is a **label selector**, NOT the log itself.

What it means:

- **app="backend"** → logs belong to backend application
- **container="auth-service"** → logs came from the auth-service container

Loki first filters by **labels**, then shows logs.

Filtering Inside the Logs:

Once labels select the stream, you can filter log content.

```
{app="backend", container="auth-service"} |= "error"
```

Output

```
2026-01-30 10:01:15 Invalid password for user admin
```

Part	Example	Purpose
Labels	<code>{app="backend"}</code>	Fast filtering
Log line	<code>Invalid password</code>	Human-readable
Indexed?	Labels only	Saves cost

1. How does Loki know which labels to use?
2. Does Loki auto-decode labels or does the user add them?
 - **Loki does NOT automatically decide labels.**
 - **Labels are added by Promtail (or the user via configuration).**

Key line to remember:

Loki only stores and queries labels. Promtail creates them.

Component	Responsibility
Docker	Produces logs (stdout/stderr)
Promtail	Reads logs + adds labels
Loki	Stores logs using labels
Grafana	Queries logs using labels

There are **3 main sources of labels**

1. Auto Labels (from Docker)

Promtail can automatically extract metadata from Docker.

Examples:

```
container_name="web-app"
container_id="a1b2c3"
image="node:18"
```

These come from **Docker itself**, not Loki.

2. User-Defined Labels (MOST IMPORTANT)

- You define labels in **Promtail configuration**.
- You manually add these labels.

3. Labels Extracted from Log Content (Advanced).

Promtail can parse the log message and extract fields as labels.

Important Rule (Interview GOLD)

Labels should be LOW-CARDINALITY

Does Loki automatically generate labels?

No. Labels are generated by Promtail or explicitly configured by the user.

Where are labels configured?

In Promtail's configuration file under **scrape_configs** and **pipeline_stages**.

Can Loki work without labels?

No. Loki fundamentally depends on labels to organize and query logs.

Can we change labels after logs are stored?

No. Labels are immutable once logs are ingested.

App → prints logs

Docker → captures stdout/stderr

Promtail → ships logs

Loki → stores logs

Grafana → shows & alerts

No need to install loki, promtail, geaphana if you use docker it will run on docker

✓ Correct way (THIS IS THE FIX)

Grafana must talk to Loki using Docker service name

✓ Loki URL inside Grafana must be:

```
arduino
```

 Copy code

```
http://loki:3100
```

NOT:

```
cpp
```

 Copy code

```
http://127.0.0.1:3100
```

How to fix it (step-by-step)

1 Open Grafana UI

```
arduino
```

 Copy code

```
http://localhost:3002
```

Docker Compose creates an **internal DNS**:

css

 Copy code

```
loki → Loki container IP
grafana → Grafana container IP
```

So inside Docker network:

nginx

 Copy code

```
Grafana → http://loki:3100 → Loki
```

That's the **correct container-to-container communication**.

Mental picture (important)

nginx

 Copy code

HOST

- └─ localhost:3100 → Loki container ✓
- └─ localhost:3002 → Grafana container ✓

DOCKER NETWORK

- └─ grafana → http://loki:3100 ✓
- └─ grafana → http://127.0.0.1:3100 ✗

Explain example Files

 **Docker Compose Stack Explained.pdf**

 **Promtail Configuration Explained.pdf**